

Telecom Network Operations & Intelligence Platform (NOC)

Complete Operations & Execution Guide

This document provides all commands, execution steps, architectural details, and troubleshooting procedures for running and managing the **Telecom Network Operations & Intelligence Platform**.

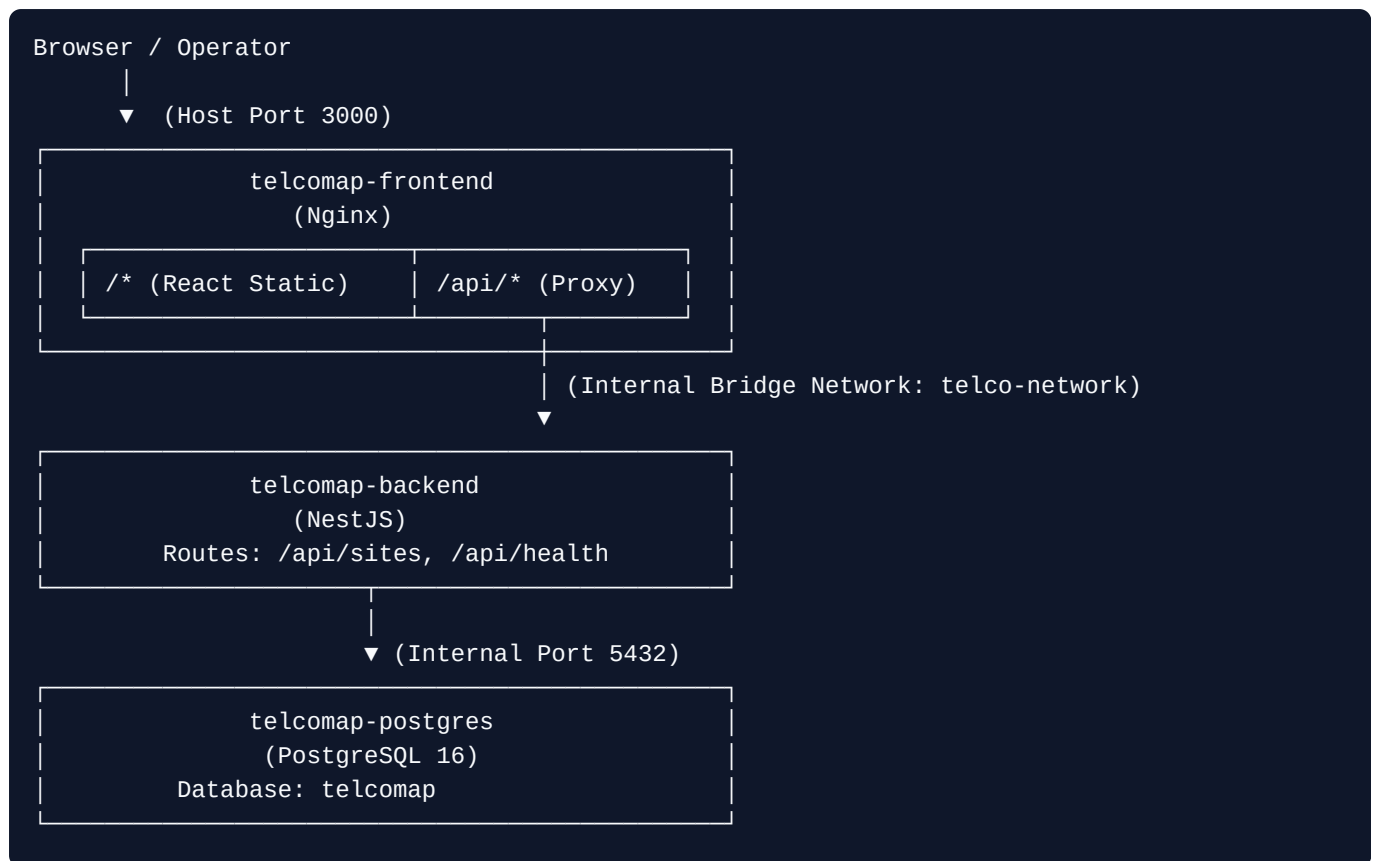
Table of Contents

- 1. [System Architecture & Port Allocation](#)
- 2. [Prerequisites](#)
- 3. [Quick Start \(Docker Compose\)](#)
- 4. [Container Management Commands](#)
- 5. [Database Operations & Prisma Seeding](#)
- 6. [Local Development \(Without Docker\)](#)
- 7. [API Verification & Testing Endpoints](#)
- 8. [NOC Web Dashboard Features](#)
- 9. [Troubleshooting & Common Questions](#)

1. System Architecture & Port Allocation

The platform runs as a coordinated multi-container service mesh:

Service Name	Container Name	Internal Port	Host Port	Role
frontend	telcomap-frontend	80	3000	Nginx serving React SPA & reverse proxying /api/*
backend	telcomap-backend	3001	3001	NestJS REST API with Prisma ORM
postgres	telcomap-postgres	5432	5433	PostgreSQL 16 Relational Database



2. Prerequisites

Ensure the following tools are installed on your host machine:

- **Docker Engine:** Version 24+ (`docker --version`)
- **Docker Compose:** Version 2.20+ (`docker compose version`)
- **Node.js:** Version 20+ (*Optional, only needed if running outside Docker*)
- **curl** or **wget:** For CLI endpoint verification

3. Quick Start (Docker Compose)

The easiest and recommended way to start the entire platform is with Docker Compose.

Step 1: Navigate to Project Directory

```
cd /home/cis/TelcoMap
```

Step 2: Configure Environment Variables

If `.env` does not already exist, copy from the example template:

```
cp .env.example .env
```

Step 3: Build and Start Containers

```
docker compose up --build -d
```

Step 4: Verify Container Health

```
docker compose ps
```

*All three services (`telcomap-postgres` , `telcomap-backend` , `telcomap-frontend`) should display status **Up (healthy)**.*

Step 5: Open the Platform in Your Browser

Open your browser and navigate to: 🖱️ <http://localhost:3000>

4. Container Management Commands

View Real-Time Logs

```
# View backend application logs
docker compose logs -f backend

# View frontend / Nginx access logs
docker compose logs -f frontend

# View database logs
docker compose logs -f postgres

# View all container logs combined
docker compose logs -f
```

Restart Services

```
# Restart entire platform
docker compose restart

# Restart only backend
docker compose restart backend
```

Stop Containers

```
# Stop containers without losing database data
docker compose down

# Stop containers AND delete database volume (fresh reset)
docker compose down -v
```

5. Database Operations & Prisma Seeding

The database is automatically initialized and seeded with 5 multi-technology sites and 10 hardware devices on initial container boot.

Re-run Database Schema Push

```
docker compose exec backend npx prisma db push
```

Re-seed the Initial Network Data

```
docker compose exec backend node dist/prisma/seed.js
```

Access PostgreSQL Interactive Shell (psql)

```
docker compose exec postgres psql -U postgres -d telcomap
```

Helpful SQL commands once inside `psql`:

```
-- View all sites and their operational status
SELECT "siteCode", "siteName", "city", "primaryTech", "backupTech", "status" FROM "Site";

-- View all devices and their assigned site
SELECT d."deviceCode", d."name", d."vendor", d."status", s."siteCode"
FROM "Device" d
JOIN "Site" s ON d."siteId" = s."id";

-- Exit psql
\q
```

6. Local Development (Without Docker)

You can run the backend and frontend locally for development.

1. Start PostgreSQL

You can keep PostgreSQL running in Docker:

```
docker compose up -d postgres
```

2. Run Backend Locally

```
cd /home/cis/TelcoMap/backend

# Install dependencies
npm install

# Generate Prisma Client
npx prisma generate

# Apply schema & seed database
npx prisma db push
node dist/prisma/seed.js

# Start backend in watch mode (listens on http://localhost:3001)
npm run start:dev
```

3. Run Frontend Locally

In a separate terminal:

```
cd /home/cis/TelcoMap/frontend

# Install dependencies
npm install

# Start Vite dev server (listens on http://localhost:3000)
npm run dev
```

7. API Verification & Testing Endpoints

Test all API endpoints directly through the Nginx reverse proxy on port 3000:

1. System Health Check

Verifies backend service uptime and live PostgreSQL connectivity:

```
curl -s http://localhost:3000/api/health | jq .
```

Expected Response:

```
{
  "status": "ok",
  "service": "telecom-network-backend",
  "timestamp": "2026-09-23T08:56:13.675Z",
  "uptimeSeconds": 16,
  "database": "connected",
  "environment": "production"
}
```

2. Network Operational Summary

Aggregated metrics across sites, operational states, and devices:

```
curl -s http://localhost:3000/api/sites/summary | jq .
```

Expected Response:

```
{
  "totalSites": 5,
  "healthySites": 3,
  "degradedSites": 1,
  "criticalSites": 1,
  "totalDevices": 10,
  "onlineDevices": 7,
  "networkAvailabilityPercent": 70
}
```

3. List All Network Sites

```
curl -s http://localhost:3000/api/sites | jq '.[0]'
```

Expected Response:

```
{
  "siteCode": "ZM-001",
  "siteName": "Lusaka Central Hub",
  "city": "Lusaka",
  "status": "HEALTHY",
  "primaryTech": "FIVE_G",
  "backupTech": "FIBRE",
  "devices": [
    {
      "deviceCode": "DEV-ZM001-01",
      "name": "Lusaka 5G Baseband Gateway",
      "vendor": "Huawei",
      "status": "ONLINE"
    }
  ]
}
```

4. Filter Sites by Operational Status

```
# Query only degraded sites
curl -s "http://localhost:3000/api/sites?status=DEGRADED" | jq .

# Query only critical sites
curl -s "http://localhost:3000/api/sites?status=CRITICAL" | jq .
```

5. Network Topology Hierarchy

```
curl -s http://localhost:3000/api/sites/topology | jq '{nodesCount: (.nodes | length), edgesCount: (.edges | length)}
```

Expected Response:

```
{
  "nodesCount": 20,
  "edgesCount": 19
}
```

8. NOC Web Dashboard Features

When you open `http://localhost:3000` in your browser, you have access to three unified operational views:

1. Top Header:

- Platform branding with pulsing NOC radar icon.
- Real-time backend API & PostgreSQL connection status pill.
- Manual data refresh button.
- View navigation buttons (`NOC Map` , `Topology` , `Inventory`).

2. Executive Metric Cards:

- **Total Sites:** `5` multi-technology locations.
- **Healthy:** `3` sites operating normally (Green).
- **Degraded:** `1` site with high latency/jitter (Amber).
- **Critical:** `1` site experiencing major link loss (Red with pulsing alert).
- **Devices:** `7 / 10` online hardware components.
- **SLA Index:** `70%` core network availability.

3. View 1: Geographic NOC Map:

- 100% Free Open-Source OpenStreetMap tiles (No API key or external account needed).
- Theme toggle on the map: **NOC Dark Mode** (custom CSS filter) or **Standard OSM** (full-color street view).
- Interactive pins colored according to operational health.
- Animated pulsing radar ring on critical site (`ZM-004`).
- Clicking any pin centers the map and displays the **Site Inspector** showing location, primary/backup technologies, and installed hardware.

4. View 2: End-to-End Network Topology:

- Visual architectural diagram showing:
 - **Tier 1:** Global Internet & National Core Gateway.
 - **Tier 2:** Transport Backbones (Fibre, Microwave, Satellite).
 - **Tier 3:** Regional Sites and edge routers/terminals.

5. View 3: Site Inventory Table:

- Tabular view of all sites with filter pills (`ALL` , `HEALTHY` , `DEGRADED` , `CRITICAL`).

- One-click inspection of any site.

9. Troubleshooting & Common Questions

Issue 1: "Cannot connect to the Docker daemon"

Cause: Docker desktop context is active instead of default system socket. **Fix:**

```
docker context use default
```

Issue 2: Port Conflict on Port 3000, 3001, or 5433

Cause: Another local service is using one of the mapped host ports. **Fix:** Open `.env` and change the conflicting port:

```
FRONTEND_PORT=3005  
BACKEND_PORT=3006  
DB_HOST_PORT=5434
```

Then restart:

```
docker compose up -d
```

Issue 3: How to Reset Everything to a Clean State

Fix:

```
docker compose down -v  
docker compose build --no-cache  
docker compose up -d
```